🚦 Traffic Light Detection & Driving Assistant

A computer vision mini-project built using Python, OpenCV, and Flask. This project detects traffic lights from images, videos, and live webcam feeds in real-time. It acts as an intelligent driving assistant that analyzes the scene and outputs driving commands: **STOP**, **GET READY**, or **MOVE AHEAD**.

Project Architecture & File Breakdown

The project follows a clean, modular structure. Below is a detailed, file-by-file, function-by-function breakdown of the logic.

---

1. `detector.py` (Core Computer Vision Engine)

This file handles the image processing, color thresholding, and shape filtering to isolate traffic lights from the rest of the scene.

A. Preprocessing (`enhance_image`)

Before we detect colors, we need to handle varying real-world lighting conditions (e.g., glare, nighttime, cloudy skies). We use **CLAHE** (Contrast Limited Adaptive Histogram Equalization) on the Lightness channel of the LAB color space.

```
def enhance_image(bgr_image: np.ndarray) -> np.ndarray:
    lab = cv2.cvtColor(bgr_image, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    cl = clahe.apply(l)
    return cv2.cvtColor(cv2.merge((cl, a, b)), cv2.COLOR_LAB2BGR)
```

- **Logic**: We split the image into Lightness (L) and color components (A, B). By applying CLAHE only to the Lightness channel, we increase local contrast without distorting the actual colors (which would ruin our HSV thresholding).

B. Color Thresholding (`create_mask`)

We convert the image to HSV (Hue, Saturation, Value) space and extract specific color ranges.

```
HSV_RANGES = {
    "Red": [(np.array([0, 120, 100]), np.array([10, 255, 255])), ...],
    "Yellow": [(np.array([18, 150, 160]), np.array([32, 255, 255]))],
    "Green": [(np.array([40, 80, 100]), np.array([85, 255, 255]))],
}

def create_mask(hsv_image: np.ndarray, color: str) -> np.ndarray:
    ...
    # Thresholding
    for lower, upper in ranges:
        mask |= cv2.inRange(hsv_image, lower, upper)

    # Morphological Cleanup (Opening to remove noise, Closing to fill hol
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel_small, iteratior
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel_large, iteratic
```

- **Logic**: Hue represents the actual color. Saturation and Value are kept high to only isolate *lit*, bright objects (ignoring dull painted objects). We use `cv2.inRange` to create a binary mask (white where the color exists, black otherwise). We then use Morphological Operations (`cv2.MORPH_OPEN` and `cv2.MORPH_CLOSE`) to remove tiny noise pixels and fill small holes inside actual traffic lights.

C. Shape Filtering (`find_detections`)

Color alone isn't enough (e.g., a yellow road sign or a red car). We must filter the detected blobs based on shape properties.

```
def find_detections(mask: np.ndarray, color: str, image_area: int):
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APF

    for cnt in contours:
        area = cv2.contourArea(cnt)
        # 1. Size filter (Reject tiny noise)
        if area < min_area: continue

        # 2. Circularity filter (Reject non-circular objects like signs/a
        circularity = (4 * np.pi * area) / (perimeter * perimeter)
        if circularity < 0.50: continue

        # 3. Aspect Ratio (Bounding box should be roughly square)
        x, y, w, h = cv2.boundingRect(cnt)
```

```
        aspect_ratio = max(w / h, h / w)
        if aspect_ratio > 1.6: continue
```

- **Logic**: We find the outlines (contours) of our color masks. We reject contours that are too small, not circular (using the $4 \pi A / P^2$ formula), or too rectangular (aspect ratio > 1.6). This guarantees we only detect glowing orbs of light.

---

2. `commands.py` (Driving Assistant State Machine)

When dealing with a live video feed, raw detections can flicker frame-by-frame. This file implements a temporal smoothing engine to provide stable driving commands.

```
class CommandEngine:
    def __init__(self, stability_frames: int = 3):
        self.stability_frames = stability_frames
        self.history = collections.deque(maxlen=stability_frames)
        self.current_state = "none"

    def update(self, detected_color: Optional[str], confidence: float) ->
        # Append the new frame's color to history
        self.history.append(detected_color or "none")

        # Only change state if all recent frames agree (temporal smoothir
        if len(self.history) == self.stability_frames and len(set(self.hi
            stable_color = self.history[0]
            if stable_color != self.current_state:
                self.current_state = stable_color
                self.state_start_time = time.time()
```

- **Logic**: `stability_frames = 3` means that the detector must see the *exact same* color for 3 consecutive frames before the UI updates the driving command. This prevents UI flickering if a single frame drops a detection due to camera motion blur.

---

3. `flask_app.py` (Backend Server & API)

This file sets up the Flask web server, serves the UI, and provides the API endpoints for processing images and video streams.

A. Image Detection Endpoint

```python
@app.route("/api/detect", methods=["POST"])
def detect_image():
    # 1. Parse uploaded image (multipart form data)
    file = request.files["image"]
    image = load_image(file.read())

    # 2. Run the detector
    result = detect(image)

    # 3. Return base64 encoded images and statistics as JSON
    return jsonify({
        "dominant_color": result.dominant_color,
        "annotated_image": encode_image_base64(result.annotated_image),
        "stats": compute_stats(result.detections),
        ...
    })
```

- **Logic**: Receives an image, passes it to `detector.py`, and returns the annotated image (with bounding boxes) and statistics back to the frontend.

B. Video Streaming Endpoint

```python
@app.route("/api/stream_video")
def stream_video():
    path = request.args.get("path")

    def generate():
        cap = cv2.VideoCapture(path)
        while cap.isOpened():
            ret, frame = cap.read()

            # Run detection & update command engine
            result = detect(frame)
            cmd = command_engine.update(result.dominant_color, ...)

            # Encode frame to JPEG
            _, buffer = cv2.imencode('.jpg', result.annotated_image)

            # Yield as multipart MJPEG stream
            yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' + buffe
            time.sleep(0.033) # Simulate 30 FPS
```

```
    return Response(generate(), mimetype='multipart/x-mixed-replace; bour
```

- **Logic**: Opens a video file using OpenCV. Processes the video frame-by-frame using `detector.py` and `commands.py`. It uses a generator to constantly stream the annotated frames to the browser using the `multipart/x-mixed-replace` content type (standard for MJPEG streaming).

---

4. `templates/index.html` (Frontend UI)

The HTML file handles the user interface. It contains a lot of CSS for the "premium dark mode" aesthetic, and vanilla Javascript to handle API interactions.

- **Tabs**: Logic to switch between Image, Live Camera, and Video File modes.
- **Webcam Logic**: Uses the browser's `navigator.mediaDevices.getUserMedia` to access the local camera. It draws the webcam video to an invisible `<canvas>`, converts the canvas to a base64 JPEG, and sends it to `/api/detect_frame` every 200ms.
- **Video Upload**: Uses standard HTML5 Drag & Drop API. Uploads the file via `FormData` to `/api/upload_video`, receives the stream URL, and assigns it to an `<img>` tag to display the MJPEG stream.

---

5. `utils.py` (Helper Functions)

Contains minor helper functions to keep the core files clean.

- `load_image(file_bytes)` : Safely decodes byte arrays into OpenCV BGR numpy arrays.
- `compute_stats(detections)` : Aggregates multiple bounding boxes and calculates total pixel area and maximum confidence for the frontend statistics panel.

🚀 How to Run the App

1. **Install dependencies:**

```
pip install -r requirements.txt
```

2. **Start the server:**

```
python3 flask_app.py
```

3. **Open your browser** and navigate to `http://localhost:5001`.